

A Generative Small-Model Ensemble for Clinical Span Extraction

NBME, Score Clinical Patient Notes · CA6125 LLM and RAG · NTU MSc Applied AI · May 2026

1. Author Contributions

This report was prepared collaboratively by the team listed below. Specific contributions are summarized in the table; per-rubric peer-review forms have been completed separately.

Member	Student ID	Primary contribution
Nathanael Brian	G2509095K	System design: Phase 1 retrieval-augmented pseudo-labeling pipeline; Phase 2 QLoRA fine-tuning of all four models; Phase 3 vLLM inference and majority voting; engineering pivot from V1 to V2; final submission management.
chenzi	G2509514F	Traditional ML baseline (TF-IDF + Logistic Regression with hand-crafted feature variant); DeBERTa-v3-base reference baseline; baseline submission management.
Liu Zhenzhi	G2509476H	§6 Comparison with Traditional Machine Learning; analysis of the bag-of-features representation gap.
Gao Ruixiang	G2509500J	Pipeline/ablation organization support; presentation video lead (script writing, slide deck design, embedding of real experimental figures, recording coordination); cross-check of all PPT facts against the report for consistency.
Zhang Xinying	G2509314B	Report writing coordination and integration; §2 Abstract; §3 Introduction; §4 Challenges; §5 Methodology; §7 Experimental Study; §8 Discussion; §9 Limitations; §10 Conclusion.

2. Abstract

The NBME Score Clinical Patient Notes competition asks systems to identify verbatim text spans within free-text patient notes that express specified clinical features. The dominant approach across the public Kaggle leaderboard frames this as token classification with DeBERTa variants and BIO label decoding. We take a deliberately orthogonal approach, aligned with the generative LLM paradigm covered in this course: we treat span extraction as constrained text generation using small language models, and we use retrieval-augmented generation as a data-augmentation engine rather than as an inference-time component.

Our pipeline runs in three stages. First, we use Qwen3-8B with FAISS-retrieved few-shot examples to pseudo-label approximately 29,600 (note, feature) pairs drawn from a pool of unannotated notes. Second, we fine-tune four small language models — Qwen3-1.7B, Qwen3-8B, LFM2.5-1.2B, and Llama-3.1-8B — with QLoRA on the combined original and pseudo-labeled data, using a regularised configuration (LoRA rank 8, attention-only target modules, 50% augmentation ratio, early stopping on validation loss) chosen specifically to limit overfitting on noisy pseudo-labels. Third, we serve all four adapters natively through vLLM 0.17.1, with per-note XGrammar regex constraints that prevent character-level hallucinations, and we ensemble predictions via character-level majority voting — a scheme that handles partial-overlap predictions more gracefully than span-level voting or probability averaging.

On the Kaggle public leaderboard, our four-model ensemble achieves character-level F1 of 0.732 (private leaderboard: 0.73201), compared to a DeBERTa-v3-base baseline at 0.813 and a TF-IDF + Logistic Regression baseline at 0.140. While our system underperforms the discriminative DeBERTa baseline, it substantially exceeds the lexical baseline and demonstrates that the generative paradigm — when combined with structured-output decoding

and character-level ensembling — is a viable approach to span extraction without a per-token classification head. Ablations show that QLoRA fine-tuning contributes the largest gain over few-shot-only inference (+0.222 F1 on the strongest single model), while the four-model ensemble adds a further +0.015 F1 over the strongest single model. We discuss the architectural and engineering trade-offs that explain the gap relative to DeBERTa, including the inherent inefficiency of autoregressive decoding for short-span tasks and the absence of position-aware, disentangled attention.

Keywords: span extraction, retrieval-augmented generation, small language models, QLoRA, ensemble methods, structured output, clinical NLP.

3. Introduction and Competition Description

3.1 The clinical scoring problem

The National Board of Medical Examiners (NBME) administers standardized assessments for medical licensure in the United States. One component of this assessment evaluates how well students elicit and document patient histories: after interviewing standardized patients, students write free-form clinical notes, and graders then score these notes against a rubric of clinical features that each note should express. This grading process is labor-intensive and inherently subjective, and automating it requires a system that can locate the specific text where a student documented each rubric feature.

The NBME Score Clinical Patient Notes Kaggle competition formalizes this task as a span extraction problem. Each test instance pairs a patient note with a target clinical feature drawn from the case rubric (e.g., “diminished appetite”), and the system must return the exact verbatim character offsets in the note that express that feature. Because medical students write in heterogeneous prose, a single feature may be expressed multiple times, in disjoint passages, using paraphrase rather than canonical terminology — for instance, “diminished appetite” may appear in a note as “eating less” or “clothes fit looser.” Systems must therefore locate semantically equivalent expressions of the target feature, not merely surface-form matches.

3.2 Input format, output format, and evaluation

Each row of the dataset contains four fields: (i) `pn_history`, the free-text patient history written by the medical student; (ii) `feature_text`, the clinical concept being scored, drawn from the rubric; and (iii) `case_num` and `feature_num`, identifiers for the clinical case and rubric feature. The expected output for each row is a location string containing space-separated character offset pairs, semicolon-delimited — for example, “10 25;40 55” denotes two spans: characters 10 through 25 and characters 40 through 55 of the note. The string is empty if the feature is absent from the note.

The dataset spans ten clinical cases, with approximately 1,000 fully annotated patient notes producing 9,901 (note, feature) annotation pairs in total, and a much larger pool of approximately 41,000 unannotated notes that we exploit for data augmentation in our methodology (see §5.3). Each case has, on average, 14.8 scored features, so each note generates roughly 14.8 (note, feature) prediction pairs at evaluation time.

Evaluation uses micro-averaged character-level F1, computed globally across every (note, feature) pair in the test set: each predicted character offset is compared against the ground-truth offsets at the character level; precision and recall are aggregated across all characters in all rows and combined into F1. Top public leaderboard scores in this competition reach approximately 0.90 or higher, and the highest-performing published systems exceed 0.96 [Xu et al., 2024].

3.3 Our contribution at a glance

This report describes a generative span-extraction pipeline built around a four-model ensemble of small language models: Qwen3-1.7B, Qwen3-8B, LFM2.5-1.2B, and Llama-3.1-8B. The pipeline has three sequential stages — retrieval-augmented pseudo-labeling, parameter-efficient fine-tuning with QLoRA, and constrained inference followed by character-level

majority voting — and is described in detail in §5. Section §4 lays out the principal challenges of this task that motivate our specific design choices. Section §6 contrasts our approach with a traditional bag-of-features baseline (TF-IDF + Logistic Regression) developed by our team, and §7 reports the experimental results, including ablations that isolate the contributions of fine-tuning and ensembling. We deliberately do not attempt to reproduce the DeBERTa-based pipelines that dominate the leaderboard; rather, we use a DeBERTa baseline run by our team as a reference point for honest comparison and discussion.

4. Challenges of the Problem

Before describing our solution, we identify six principal technical challenges of clinical span extraction on this dataset. Each challenge motivates a specific design choice in §5; the mapping between challenges and solution components is summarized at the end of this section.

4.1 Lexical-semantic mismatch between feature and note

The most fundamental difficulty of this task is that the rubric feature and the note rarely share surface vocabulary. A feature labeled “diminished appetite” appears in real student notes as “eating less,” “food doesn’t taste good,” or “clothes fit looser.” A feature labelled “family history of MI” appears as “Father had a heart attack at 52,” or as the abbreviated “FHx: CAD.” Any system relying on lexical overlap will miss most positive examples and predict spuriously on words that happen to coincide with the feature description. This is the dominant failure mode of the TF-IDF + LR baseline at 0.140 F1 (§6); it motivates our use of pretrained LLMs that encode paraphrase equivalence in a shared semantic space.

4.2 Multiple disjoint spans per (note, feature) pair

A single feature is often expressed in multiple non-contiguous passages within the same note. For “tobacco use,” a note may contain both “smokes half a pack per day” in the social-history paragraph and “20 pack-year history” in a later risk-factor list. Both spans are valid annotations and the system must return them as a set, not as a single answer. This rules out classifier formulations that pick one most-likely span per query, and motivates our generative output schema {“spans”: [...]} — a list-valued output that handles multi-span cases natively (§5.2).

4.3 Boundary ambiguity

It is often genuinely unclear where a span should begin or end. For the feature “dyspnea on exertion,” a note saying “Patient reports severe dyspnea on exertion for the past 2 weeks” admits at least three plausible annotations: “dyspnea on exertion,” “severe dyspnea on exertion,” and “severe dyspnea on exertion for the past 2 weeks.” Different human annotators legitimately disagree on the right scope, and the character-level F1 metric penalizes systematic over- or under-prediction at the boundaries. This motivates the character-level majority voting in §5.5: when individual ensemble members disagree on a few characters at the boundary, the consensus across four models recovers the agreed-upon core of the span and discards positions where agreement is partial.

4.4 Class imbalance and the empty-span case

Many features are absent from any given note — a single patient note typically expresses only a subset of the roughly 14.8 features applicable to its case. A trivial baseline that always predicts an empty span achieves perfect precision-of-empty and zero recall-of-positive, dragging F1 to near zero. Conversely, a model that hallucinates spans liberally inflates recall but is heavily penalized on precision. We address this by training on both positive and empty-span examples, with the assistant target {“spans”: []} used explicitly for the negative class — the model learns when not to extract anything (§5.4). The 3/4 vote threshold in §5.5 further raises the precision floor at inference time.

4.5 Limited labeled data relative to model scale

The competition supplies 9,901 labeled (note, feature) pairs. This is modest in absolute terms; for fine-tuning 1B–8B-parameter language models, the labeled set is roughly an order of magnitude smaller than in a standard supervised regime. Two consequences follow. First, the labeled-only signal is unlikely to fully specialize an 8B base model; we therefore expand it with retrieval-augmented pseudo-labeling, producing approximately 29,600 additional pairs (§5.3). Second, fine-tuning hyperparameters must be carefully regularised against memorizing the limited (and partially noisy) signal; we use rank-8 attention-only LoRA, a conservative learning rate of 1×10^{-4} , and early stopping on validation loss (§5.4).

4.6 Clinical domain specificity

Clinical notes contain medical abbreviations, drug names, anatomical references, and shorthand conventions that are rare in general web-text pretraining. “SOB” denotes shortness of breath in this domain, not the colloquial English meaning; “pt” denotes patient, not the unit pint; “ETOH” denotes alcohol. A model trained only on general web text may misinterpret these. Instruction-tuned LLMs such as Qwen3 and Llama-3.1 have seen some medical text during pretraining and instruction tuning, but coverage is uneven; we further compensate by injecting domain-specific examples at inference time through FAISS-retrieved few-shot demonstrations (§5.3).

4.7 Mapping from challenges to solution components

Table 1 summarises which subsection of §5 addresses each challenge identified above.

Challenge	Solution component	Section
Lexical-semantic mismatch	Pretrained LLM with semantic representations	§5.4 (base models)
Multiple disjoint spans	Generative JSON-list output	§5.2
Boundary ambiguity	Character-level majority voting across 4 models	§5.5
Class imbalance / empty-span case	Joint positive/negative training; vote threshold $\geq 3/4$	§5.4, §5.5
Limited labeled data	RAG-based pseudo-labeling; conservative LoRA + early stopping	§5.3, §5.4
Clinical domain specificity	FAISS-retrieved few-shot examples; per-note FSM constraint	§5.3, §5.5

Table 1. Mapping from problem-level challenges (§4) to solution-level components (§5).

5. Proposed Solution: A Generative Small-Model Ensemble for Span Extraction

5.1 System overview and design rationale

The competition task is to identify, within each free-text patient note (pn_history), the verbatim character spans that express the target clinical feature (feature_text). Almost every top-ranked solution on the public Kaggle leaderboard frames this as a token classification problem, solved with DeBERTa using BIO labels, followed by span decoding from contiguous label runs. While reproducing this paradigm would have been straightforward, we judged that it would offer limited educational value for a course centered on large language models and retrieval-augmented generation: the dominant DeBERTa pipeline does not exercise any of the techniques that distinguish the recent generative LLM stack from earlier encoder-only NLP.

We therefore pursue a deliberately orthogonal solution. We frame span extraction as a constrained text generation problem in which a small language model reads a (note, feature) pair and emits a JSON object listing the verbatim spans. We use retrieval-augmented

generation not at inference time — the standard RAG pattern — but as a data augmentation engine that produces additional training examples from a large unlabeled note pool. We then fine-tune a heterogeneous ensemble of four small language models with QLoRA on the union of the original and pseudo-labeled data, and serve them via vLLM with per-note regex constraints to prevent character-level hallucinations. Final predictions are obtained by character-level majority voting across the four models.

This design explicitly exercises four pillars of the course content: prompt engineering and few-shot in-context learning (Phase 1), parameter-efficient fine-tuning (Phase 2), structured-output decoding (Phase 3), and ensemble methods. The pipeline is summarized in Figure 1 and elaborated in the following subsections.

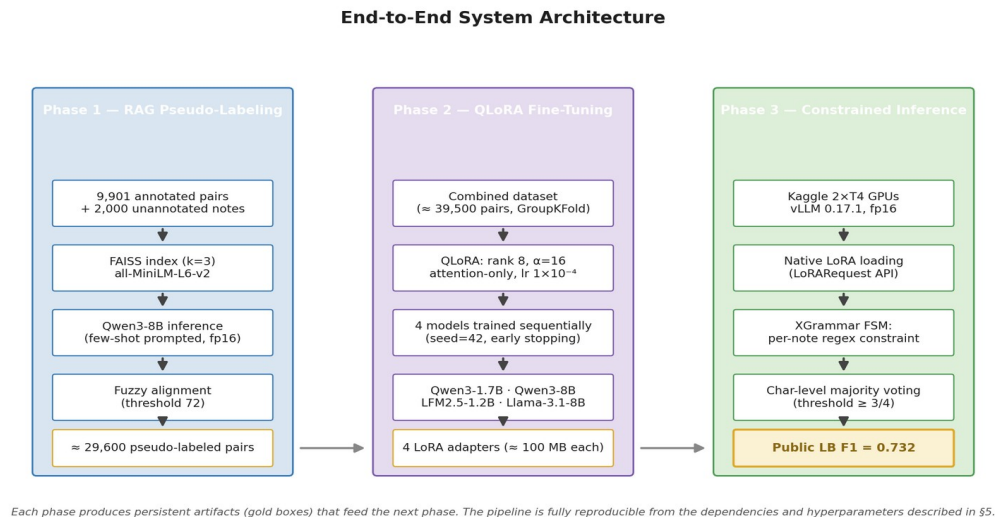


Figure 1. End-to-end system architecture: Phase 1 (RAG-based pseudo-labeling with Qwen3-8B), Phase 2 (sequential QLoRA fine-tuning of four small language models), and Phase 3 (constrained inference plus character-level majority voting).

5.2 Why a generative formulation

A generative formulation has three properties that motivated our choice, and one cost we accepted up front. Native handling of multiple disjoint spans: a clinical feature such as “diminished appetite” may be expressed several times in a single note, in passages such as “eating less” or “clothes fit looser.” A token classification model handles this by emitting a B-label at each new span and decoding contiguous B/I runs. A generative model handles it by emitting a list of strings. The generative output is structurally simpler and avoids the BIO-decoding heuristics that are a known source of boundary errors in the discriminative pipeline.

No per-token classification head: a small generative model can be fine-tuned with a single adapter and reused for any new task by changing only the prompt. Adding a new clinical-feature taxonomy does not require retraining a classification head with different output dimensions, which is a practical advantage for the kind of evolving rubrics that real medical assessment systems face.

Compositional benefits with retrieval and structured output: few-shot prompting with retrieved examples and grammar-constrained decoding compose naturally with the generative interface. Both are awkward to retrofit onto a token classifier, and both are core techniques in the modern LLM toolbox.

The cost we accepted: inference is slower, and hallucination is a risk. Autoregressive decoding emits one token at a time and is inherently slower than a single forward pass through a token classifier. The model can also emit characters that do not appear in the source note. We explicitly mitigate the risk of hallucinations through the per-note regex finite-state machine

described in §5.5, and we reduce latency costs by using small models and tensor parallelism rather than a single large model.

5.3 Phase 1 — Retrieval-augmented pseudo-labeling

The competition supplies approximately 1,000 fully annotated patient notes (yielding 9,901 (note, feature) annotation pairs) and a much larger pool of approximately 41,000 unannotated notes. Phase 1 uses a strong off-the-shelf model — Qwen3-8B in fp16 — to expand the labeled set by pseudo-annotating 2,000 of these unannotated notes. Each note is paired with each of the roughly 14.8 features applicable to its clinical case, producing approximately 29,600 additional training pairs.

The challenge in pseudo-labeling is that the pseudo-labeler must be steered toward the annotation conventions of the original training set without seeing the entire 9,901-pair corpus in its context. We address this with FAISS-based few-shot retrieval. Each annotated example is embedded as the string “Feature: <feature_text> Annotation: <gold_span>” using the all-MiniLM-L6-v2 sentence encoder, producing a 384-dimensional vector. The full corpus is indexed in an IndexFlatIP index over L2-normalized vectors, which corresponds to cosine similarity. At pseudo-labeling time, each target feature is embedded as “Feature: <feature_text> Annotation: —” — the annotation prefix is included but left empty, so retrieval finds training examples whose feature description is semantically closest to the query feature, and the three nearest neighbors are inserted into the prompt as in-context demonstrations. We fixed the retrieval count at $k=3$, which provided sufficient in-context diversity to capture clinical paraphrases without exceeding prompt length limits or introducing redundant examples that saturate the context window. The index is built once and cached to disk, after which subsequent runs load in seconds.

The Qwen3-8B prompt is a two-message chat: a system message instructing the model to emit only verbatim spans as a JSON object {“spans”: [...]} with no markdown or reasoning trace, and a user message containing the three retrieved examples followed by the target note and feature. We disable Qwen3’s enable_thinking mode, because the chain-of-thought traces consume the 128-token output budget and do not improve answer quality on this short-output task. We use greedy decoding (do_sample=False), a 4096-token context window, and a batch size of 32 prompts per GPU call.

Because the model emits span strings rather than character offsets, we map strings back to offsets through a three-step procedure: exact substring search, then case-insensitive search, and finally a rapidfuzz sliding-window search over substrings within $\pm 20\%$ of the target length, accepting matches with a fuzz score of at least 72.

5.4 Phase 2 — QLoRA fine-tuning of a four-model ensemble

To maximize ensemble diversity — and thus ensure members fail in different ways — we selected four models intentionally spanning two distinct architectures (Qwen3 and LFM2.5 vs. Llama-3.1) and two parameter scales (1.2B and 8B). This strategy assumes that different model families possess distinct capabilities and inductive biases shaped by their respective pretraining corpora and architectural designs, allowing them to capture complementary aspects of the clinical span extraction task. We do not rely on seed variation alone for diversity: §7.4 shows that duplicating the same fine-tuned architecture under different seeds yields no gain under our character-level voting scheme; our diversity instead comes primarily from spanning two architectures and two parameter scales.

The four models we fine-tune are listed in Table 2.

Model	Parameters	Adapter directory
Qwen3-1.7B	1.7 B	adapters/qwen3_1_7b_adapter
Qwen3-8B	8 B	adapters/qwen3_8b_adapter

Model	Parameters	Adapter directory
LFM2.5-1.2B-Instruct	1.2 B	adapters/lfm2_5_1_2b_adapter
Llama-3.1-8B-Instruct	8 B	adapters/llama3_1_8b_adapter

Table 2. Four-model ensemble registry, all fine-tuned with QLoRA.

Models are trained sequentially on a single GPU, and the previous model is fully unloaded before the next one loads. Each model is trained with a distinct random seed (Llama-3.1-8B = 42, Qwen3-1.7B = 43, Qwen3-8B = 44, LFM2.5-1.2B = 45), so ensemble diversity arises from architectural and pretraining differences combined with independent training randomness. Each model is loaded in 4-bit NF4 quantization with bfloat16 compute and double quantization. We then attach a LoRA adapter with rank 8, alpha 16, and dropout 0.10 to the attention projection modules (q_proj, k_proj, v_proj, o_proj) and the analogous projections in LFM2.5’s custom architecture. This is a deliberately conservative configuration relative to the more common rank-16/alpha-32/7-module setup. Earlier experiments with the larger configuration showed faster memorization of training labels without a proportionate improvement in generalization; restricting LoRA to attention modules and lowering its capacity acts as an implicit regularizer against memorizing the noisy pseudo-labels produced in Phase 1.

We enable gradient checkpointing and cast LayerNorm layers to float32 to ensure stable training with the quantized base weights. Training hyperparameters are uniform across the four models, both to control for hyperparameter selection bias and to keep the training pipeline maintainable: learning rate 1×10^{-4} (a deliberately conservative value, again to slow down memorisation of noisy labels), three epochs, per-device batch size 8 with gradient accumulation 2 (effective batch 16), cosine schedule with 5% warmup, weight decay 0.05, AdamW-8bit optimizer, and a maximum sequence length of 1024 tokens. We additionally enable early stopping on validation loss with patience=3 and load_best_model_at_end=True, which ensures the saved adapter corresponds to the validation-best checkpoint rather than the final (potentially overfitted) step. This is a critical implementation detail — without early stopping, all four models meaningfully overfit beyond their best checkpoint (typically reached at steps 1,000–2,500 of training; see §7.5).

A subtle but important choice is the train/validation split. We use GroupKFold(n_splits=5) over case_num rather than a row-level random split. The dataset contains only 10 clinical cases, and notes within the same case share clinical context and vocabulary; mixing cases between the train and validation sets would yield overly optimistic and uninformative validation scores. The fifth fold (two of the ten cases) is held out as validation throughout training. We further set the augmentation ratio to AUG_SAMPLE_RATIO = 0.5, using only half of the Phase-1 pseudo-labels in each training run; this trades some training-set scale for reduced exposure to noisy teacher labels.

The loss is standard causal-LM cross-entropy computed only over the assistant response tokens; the system prompt and user message are masked with label index -100 and do not contribute to the gradient. The assistant target for each example is the JSON string {"spans": ["verbatim span", ...]} for positive instances and {"spans": []} for negatives. Each fine-tuned model contributes only a roughly 100 MB LoRA adapter, not the full quantized base weights, which makes the four-adapter checkpoint compact enough to ship to the Kaggle inference environment as a private dataset.

5.5 Phase 3 — Constrained inference and character-level voting

Phase 3 runs inside a Kaggle notebook with two NVIDIA T4 GPUs (29.2 GB total VRAM, CUDA 12.8, compute capability 7.5). T4 hardware does not support native bfloat16, so all inference is in float16. CUDA 12.8 forces us to vLLM 0.17.1, the last release that compiles against this toolkit. The two 8B members of the ensemble (Qwen3-8B, Llama-3.1-8B) require tensor parallelism (tensor_parallel_size=2) across both GPUs, while the two smaller models each occupy a single GPU.

For each of the four models in turn, we load the base model into vLLM with `enable_lora=True`, `max_lora_rank=16`, and then attach the model’s LoRA adapter at request time via vLLM’s `LoRARequest` API. This avoids the merge-to-disk step that the V1 pipeline used (§5.6), saving roughly two to five minutes per model and approximately 16 GB of disk I/O over a full inference run. We set `enforce_eager=True` to ensure clean VRAM release between sequential model loads — without it, residual CUDA graph caches accumulate across adapter swaps and trigger out-of-memory failures on the constrained 29.2 GB T4 budget. We select the Triton attention backend, which is required for vLLM’s paged KV cache on T4 hardware. GPU memory utilization is capped at 0.85 to leave headroom for the small KV cache fluctuations we observed at runtime.

Per-note regex constraints. vLLM 0.17.1 supports structured-output decoding through its XGrammar backend. For each test row, we construct a regex that enforces both the JSON structure of the output (`{“spans”: [“...”]}`) and a per-note character class: each generated span string may contain only characters that appear in the source `pn_history`. XGrammar compiles this regex into a finite-state machine, and the decoder masks logits at every step so that only legal next characters can be sampled. The model, therefore, cannot hallucinate characters absent from the note, a stronger guarantee than post-hoc filtering. The constraint is specific to generative approaches: discriminative token classifiers — which dominate the public leaderboard for this competition — predict spans by labeling existing tokens, and so face no analogous hallucination problem. Among the generative solutions in this competition, we did not encounter this mechanism in the public discussion threads we reviewed, and we consider it one of the principal contributions of our approach.

Character-level majority voting. After all four models have produced their span string lists for every test row, we resolve each model’s spans to character offsets in the note. For string-to-offset resolution, we use the three-step cascade described in Phase 1 (exact → case-insensitive → fuzzy), but with a relaxed threshold: we use a fuzzy-match score of 70 in Phase 3, compared to 72 in Phase 1. This reflects a deliberate precision-recall trade-off: Phase 1 employs a stricter cutoff of 72 to prioritize precision and prevent hallucinated spans from corrupting the pseudo-labeled training signal; in Phase 3 (test-time inference), we relax to 70 to maximize recall during inference, as the character-level majority voting across four diverse models naturally filters out any spurious or imprecise boundary matches introduced by the looser alignment.

For each test row, we build four binary arrays of length `len(pn_history)`, one per model, with 1s at positions inside any predicted span. We sum the four arrays element-wise, apply a threshold of 3 (i.e., at least three of the four models must agree on a position), and extract contiguous runs of 1s as the final predicted spans. The 3/4 threshold was selected as a conservative majority consensus to minimize hallucination risks; our initial ensemble experiments began with 3 models and a 2/3 vote threshold, and adding a fourth model to evaluate marginal gains revealed that a 3/4 consensus better prevented spurious predictions while preserving recall. Character-level voting is preferable to span-level voting because it preserves partial agreement among models: if three models predict “substernal pressure” and one predicts “substernal chest pressure,” character-level voting retains the overlapping characters and recovers the consensus span.

5.6 Engineering pivot from V1 to V2

Our first inference pipeline (V1) targeted Qwen3.5-9B, Gemma-4-E4B, and Qwen3.5-4B and used a more conventional merge-to-disk LoRA strategy. It failed in the Kaggle environment for two reasons. First, vLLM versions $\leq 0.19.1$ incorrectly route Qwen3.5 and Gemma-4 to the vision-language input handler, raising a `preprocessor_config.json not found` error at runtime. Second, vLLM versions $\geq 0.20.0$ require CUDA 13, which is not yet available in the Kaggle T4 image (which has CUDA 12.8). We therefore migrated to V2: model selection was constrained to Qwen3 (not Qwen3.5), LFM2.5, and Llama-3.1; LoRA loading switched from disk-based merging to native `LoRARequest`; and structured-output decoding via XGrammar was added to harden against hallucinations.

6. Comparison with Traditional Machine Learning

The course rubric requires a comparison between our LLM/RAG-based pipeline and traditional machine learning approaches to the same task. This section presents that comparison through the lens of a TF-IDF + Logistic Regression baseline developed by our team, and discusses the benefits and disadvantages of each paradigm for span extraction in clinical notes.

6.1 The traditional ML baseline: TF-IDF + Logistic Regression

Our team’s traditional ML baseline frames the span extraction task as a candidate-level binary classification problem. Each patient note is first split into candidate text segments using punctuation and line breaks as delimiters (commas, periods, semicolons, and newlines); each candidate is then paired with the target feature description and assigned a binary label indicating whether the candidate matches one of the gold span texts for that (note, feature) pair. The model takes the concatenated text “feature: <feature_text> [SEP] candidate: <segment>” as input, vectorizes it with a TF-IDF representation (lowercased unigrams and bigrams, vocabulary capped at 50,000 features, minimum document frequency of 2), and feeds the result to a Logistic Regression classifier with class-balanced sample weights and the scikit-learn default L2 regularization. The dataset is split with a GroupShuffleSplit, holding out 20% of patient notes — grouped by pn_num — for validation. At inference time, the highest-scoring candidate is selected per (note, feature) test row, and a span is emitted only if its predicted probability exceeds 0.7; otherwise, the system predicts an empty string.

This baseline achieves a character-level F1 of 0.140 on the Kaggle public leaderboard. We additionally explored an enhanced variant that augments the TF-IDF features with ten hand-crafted matching features per (feature, candidate) pair — token-overlap counts and ratios, character-length features, and binary substring- and exact-match flags — and observed only a marginal improvement, suggesting that lexical and shallow lexical-matching features are not sufficient to bridge the gap to a semantic representation.

6.2 Benefits of traditional ML for span extraction

Traditional machine learning approaches, such as our TF-IDF + Logistic Regression baseline, offer several critical advantages in clinical contexts. First, they provide high interpretability; every feature weight in the TF-IDF vectorizer is directly inspectable, allowing clinicians to audit the exact lexical cues that triggered a span prediction — a vital requirement in regulated medical environments. Second, these models exhibit superior computational efficiency, training in seconds on standard CPUs and offering sub-millisecond inference without requiring expensive GPU hardware. Finally, they possess bounded failure modes; unlike generative models, they are restricted to the provided vocabulary and cannot “hallucinate” text or generate characters absent from the source note. Together, these traits ensure a level of deployability and safety essential for clinical settings.

6.3 Limitations of traditional ML and the case for LLM/RAG

Despite these benefits, the TF-IDF + LR baseline reaches only 0.140 F1, against our generative SLM ensemble’s 0.732 — a roughly fivefold improvement, on the same data, with the same evaluation protocol. The size of this gap is informative; it indicates a structural mismatch between the bag-of-features representation and the task’s demands.

The TF-IDF + LR baseline’s performance deficit is primarily driven by three structural limitations that our LLM/RAG approach systematically overcomes. First, the traditional model lacks semantic equivalence; because TF-IDF relies on lexical matching, it treats semantically identical phrases like “eating less” and “diminished appetite” as entirely unrelated features. In contrast, our LLMs leverage a shared semantic space to recognize these paraphrases as the same clinical concept. Second, the baseline possesses no pretrained world knowledge, meaning a freshly fitted LR model cannot know that “retrosternal” and “substernal” refer to the same anatomical region. Our pipeline addresses this through RAG-based pseudo-labeling, which leverages the LLM’s inherent world knowledge to expand the data. Third, while LR makes per-

token decisions in isolation, LLMs employ contextual reasoning by attending across the full patient note. We further specialize these pretrained representations to clinical-note conventions using QLoRA fine-tuning.

Ultimately, overcoming semantic equivalence is the dominant challenge in clinical span extraction, which is why even our smallest 1.2B-parameter LLM outperforms the TF-IDF + LR baseline by roughly fivefold. There remain real disadvantages of the LLM/RAG paradigm that we acknowledge openly: autoregressive decoding is slow relative to a single forward pass through a token classifier, the risk of hallucinating characters absent from the source note must be actively mitigated (we do so via per-note XGrammar regex constraints; see §5.5), and the parameter and compute footprint is orders of magnitude larger than the TF-IDF baseline. We discuss these tradeoffs further in §7 and §9. We note finally that our LLM-based system still underperforms a discriminative DeBERTa baseline at 0.813 (§7.6); the comparison here, however, is between two paradigms that both leverage pretrained semantic representations — the gap between TF-IDF + LR and any modern neural approach, by contrast, reflects a deeper representational divide and is of a different order of magnitude.

7. Experimental Study

7.1 Evaluation metric and validation protocol

All results are reported using the competition’s official metric: micro-averaged character-level F1 computed globally over all (note, feature) pairs in the test set. Each predicted character offset is compared against ground-truth character-level offsets; precision and recall are computed for all characters in all rows, then combined into an F1 score. This metric, rather than span-level F1, is what the Kaggle leaderboard reports and what we therefore optimize for. During development, we additionally tracked validation character-level F1 on Fold 4 of a GroupKFold($n_splits=5$) split over `case_num` (§5.4). All numbers reported below as “leaderboard F1” are from the Kaggle public leaderboard score, which is computed on approximately 80% of the held-out test set; the private leaderboard score for our final 4-model ensemble submission is 0.73201, equivalent to 0.732 at three-decimal precision and confirming the public-leaderboard figure.

7.2 Main results

Table 3 summarizes the F1 scores of the systems we compared. We include three internal team baselines (TF-IDF + LR, our pretrained-only ensemble, and a DeBERTa run), the four LoRA-fine-tuned single models, two ensemble configurations, and one external reference point from the literature.

System	Approach	F1	Source
TF-IDF + LR	Lexical bag-of-features classifier	0.140	Team baseline (chenzi)
Few-shot SLM ensemble	3 models, retrieval-augmented prompting, no fine-tuning	0.531	This work, ablation
Qwen3-1.7B + LoRA	Single-model LoRA fine-tuned	0.651	This work, ablation
LFM2.5-1.2B + LoRA	Single-model LoRA fine-tuned	0.634	This work, ablation
Llama-3.1-8B + LoRA	Single-model LoRA fine-tuned	0.690	This work, ablation
Qwen3-8B + LoRA (best single)	Single-model LoRA fine-tuned	0.717	This work, ablation
3-model LoRA	Qwen3-8B + Qwen3-1.7B +	0.710	This work, ablation

System	Approach	F1	Source
ensemble	LFM2.5, vote thr. 2		
Our 4-model LoRA ensemble	Phase 1 + Phase 2 + Phase 3 (full pipeline), vote thr. 3	0.732	This work, primary result
DeBERTa baseline	DeBERTa-v3-base, 1 epoch, BIO token classification	0.813	Team baseline (chenzi); own local training run
Top published LLM	Two-phase LLM framework	0.968–0.983	Xu et al., 2024

Table 3. End-to-end comparison of all systems evaluated on the NBME public leaderboard. The bold row indicates our final submitted system.

Three observations stand out. First, the gap between the lexical baseline (0.140) and any LLM-based system is large, even before fine-tuning: a few-shot prompted ensemble reaches 0.531, almost four times the lexical F1. This confirms that the task requires semantic matching of paraphrased medical concepts — lexical features are simply not expressive enough. Second, our final 4-model LoRA ensemble (0.732) substantially exceeds every single fine-tuned model in the table; we attribute the +0.015 gap over the strongest single model (Qwen3-8B at 0.717) to the architectural and pretraining diversity of the ensemble members, discussed in §7.4. Third, our system underperforms the DeBERTa baseline (0.813), which we treat seriously and address in §7.6.

7.3 Ablation: effect of QLoRA fine-tuning

To isolate the contribution of fine-tuning relative to in-context learning, we compared each model in two settings: pretrained with three FAISS-retrieved few-shot examples per query (§5.3-style prompting), and the same model after QLoRA fine-tuning on the combined original + pseudo-labeled dataset (§5.4). Results are shown in Table 4.

Model	Pretrained + few-shot	LoRA fine-tuned	Δ F1
Qwen3-1.7B	0.481	0.651	+0.170
Qwen3-8B	0.495	0.717	+0.222
3-model ensemble (Qwen3-1.7B + Qwen3-8B + LFM2.5)	0.531	0.710	+0.179

Table 4. Effect of QLoRA fine-tuning on individual models and on a 3-model ensemble. All values are Kaggle public leaderboard F1.

Fine-tuning contributes a consistent gain of roughly +0.17 to +0.22 F1 across both individual models and the ensemble, independent of model scale. This is by a wide margin the largest single intervention in our pipeline. The magnitude of the gain is informative: the few-shot configuration already provides the model with three labeled examples in-context, but those three examples cannot fully convey the dataset’s annotation conventions — in particular, the style of partial-span boundaries, the handling of negations, and the conventions around paraphrase. Fine-tuning over thousands of pseudo-labeled and gold examples internalizes these conventions in ways that few-shot prompting cannot replicate within a fixed context window.

We note that the absolute few-shot F1 of 0.531 is already a non-trivial result for an unfit model and supports the case that the generative formulation is viable in its own right, before any task-specific training. In an additional pretrained-only experiment, we tested an ensemble of three 7B–8B-class LLMs (Llama-3.1-8B + Qwen3-8B + Mistral-7B) and observed near-identical performance (0.538 F1) to the mixed-scale ensemble reported in Table 4 (Qwen3-1.7B + Qwen3-8B + LFM2.5-1.2B at 0.531). This suggests that, in the few-shot regime, the specific

choice of base model matters much less than after fine-tuning — consistent with the +0.17 to +0.22 fine-tuning effect being the dominant force in our pipeline.

7.4 Ablation: effect of ensembling and architectural diversity

Holding LoRA fine-tuning fixed, we examined whether stacking models in an ensemble improves character-level F1 and how the choice of ensemble members affects the results. The findings are nuanced and worth presenting in full.

Configuration	F1	Δ vs best single
Best single model (Qwen3-8B + LoRA)	0.717	—
3-model ensemble (Qwen3-8B + Qwen3-1.7B + LFM2.5)	0.710	-0.007
4-model ensemble (adds Llama-3.1-8B)	0.732	+0.015

Table 5. Effect of ensemble configuration on Kaggle public leaderboard F1. All members are LoRA fine-tuned.

Two findings worth highlighting. First, the 3-model ensemble (which excludes Llama) is slightly worse than the strongest single model. This is a useful negative result: ensembling is not automatically beneficial. Adding a comparatively weaker model to a strong one (LFM2.5-1.2B at 0.634 alongside Qwen3-8B at 0.717) can drag the consensus down when the vote threshold is calibrated for the stronger members. Second, adding a fourth model from a different lineage (Llama-3.1-8B at 0.690, comparable in strength to Qwen3-8B but architecturally and in pretraining data distinct) yields +0.022 over the 3-model variant and +0.015 over the best single model. The benefit of ensembling on this task, therefore, comes from genuine architectural and pretraining diversity rather than sheer member count. We separately observed that duplicating ensemble members (running the same fine-tuned architecture with a different seed) produced only a within-noise change (≤ 0.001 F1) under our character-level voting scheme, i.e. no meaningful gain.

7.5 Training dynamics

The Phase 2 training logs reveal a consistent pattern across all four models: training loss decreases monotonically throughout three epochs, while validation loss reaches its minimum between steps 1,000 and 2,500 and then rises steadily — clear evidence of overfitting on the noisy pseudo-label distribution. The `EarlyStoppingCallback` with `load_best_model_at_end=True` therefore loads a meaningfully different (and better) checkpoint than the final-step adapter would produce. The 8B models reach their best validation loss earlier (at step 1,000) than the 1B models (at steps 2,000–2,500), consistent with larger models having more capacity to memorize the training set more quickly. Across all four models, the JSON output rate on the validation set is 1.000, indicating that the structured output format is reliably learned.

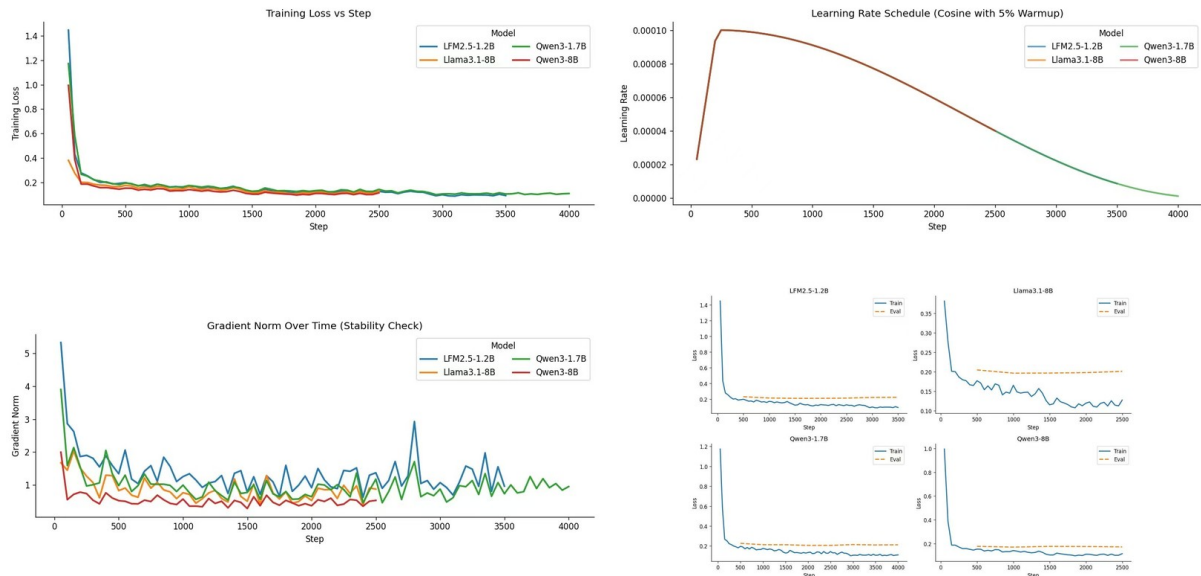


Figure 2. Phase 2 training dynamics across all four models. Top-left: training loss vs. step. Top-right: cosine learning-rate schedule with 5% warmup. Bottom-left: gradient norms over time. Bottom-right: train (solid) vs. validation (dashed) loss per model — each model’s validation-loss minimum motivates early stopping.

Figure 3 summarises the four models’ validation-time metrics in a single heatmap. Three observations are worth flagging. First, validation character-level F1 (Qwen3-8B 0.764, Llama-3.1-8B 0.711, Qwen3-1.7B 0.626, LFM2.5-1.2B 0.640) closely tracks the leaderboard ranking in Table 3, confirming that our case-grouped split provides a useful proxy for generalization. Second, all four models achieve a `val_jsonish_rate` of 1.000, indicating that the structured-output format is reliably learned regardless of the base architecture. Third, `val_contained_rate` — the fraction of predicted spans that are exact substrings of the source note — exceeds 0.93 across all four models, suggesting that our prompt and fine-tuning data already steer outputs toward verbatim extraction even before the FSM constraint described in §5.5 is applied.

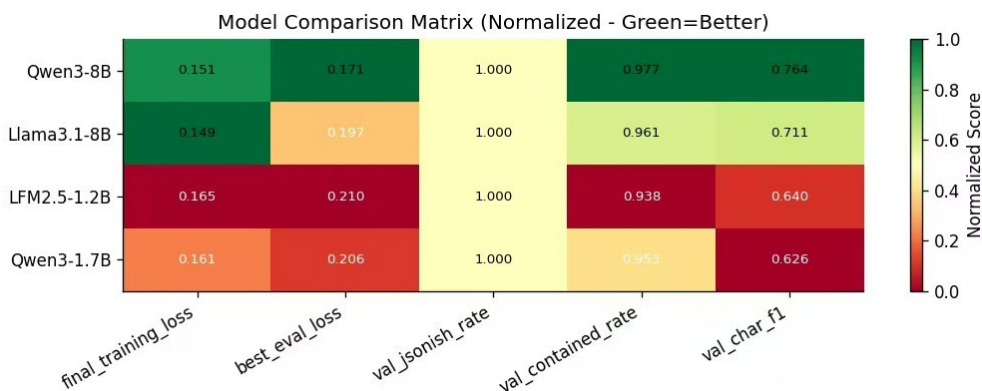


Figure 3. Validation-set metrics across the four LoRA-fine-tuned models, normalized per column (green = better). The 8B-class models (Qwen3-8B, Llama-3.1-8B) lead on every column except `val_jsonish_rate` (where all four reach 1.000); 1B-class models trail. Qwen3-8B is the strongest single model across all metrics.

7.6 Comparison with the DeBERTa discriminative baseline

Our final F1 of 0.732 is approximately 0.081 below the DeBERTa-v3-base baseline at 0.813. We treat this gap honestly rather than discount it. Three architectural and engineering reasons account for most of the difference.

Inductive bias for span tasks. DeBERTa’s disentangled attention mechanism explicitly separates content and position attention, an inductive bias that is particularly well-matched to span-extraction tasks where the boundaries between adjacent tokens carry the prediction. A decoder-only causal model has no equivalent positional inductive bias for span boundaries; it learns the same information from data, but at a higher sample cost. The fact that even DeBERTa-v3-base (~184 M parameters) outperforms our 4-model ensemble (~19 B parameters in total, with LoRA-trained adapters) underscores how much architectural fit matters at this dataset scale.

Per-token logits versus autoregressive sampling. A token classification model produces a probability distribution over BIO labels at every position in a single forward pass. Span boundaries can be tuned by thresholding these probabilities, and ensembling is a simple average over logits. Our generative pipeline produces span strings that we must then re-locate in the source text and convert to character offsets (the exact → case-insensitive → fuzzy cascade described in §5.3 and §5.5). Each of those steps is a small source of recall loss, and the cumulative effect is non-trivial.

Training-set scale and full vs. parameter-efficient fine-tuning. DeBERTa-v3-base is fine-tuned end-to-end on the 9,901 + 29,600 training pairs we have. With QLoRA, we update only the rank-8 attention adapters, leaving the frozen 4-bit base unchanged. At the dataset scale of this competition, that base distribution may not be a better fit than DeBERTa for span extraction, and full fine-tuning of an 8B base was outside the project’s compute budget.

These three reasons are not excuses for the gap: they are the substantive content of the comparison. We believe the most honest summary is that the generative paradigm is competitive but not yet superior for short-span clinical extraction, and that closing the remaining gap requires either substantially larger training data, full fine-tuning rather than QLoRA, or a hybrid generative-discriminative head — directions we discuss in §9.

7.7 Reflection: gap to top published systems

For context, top public leaderboard scores in this competition are approximately 0.90 or higher, and a recent academic paper [Xu et al., 2024] reports F1 scores of 0.968–0.983 using a two-phase LLM framework on the full dataset. Our 0.732 leaves a substantial gap to those systems. The principal differences are: (i) those systems use ensembles of 5 to 15 fully fine-tuned DeBERTa-v3-large or DeBERTa-v2-xxlarge models, often with masked language model pretraining on the clinical note corpus before downstream fine-tuning; (ii) they apply iterative pseudo-labelling rather than a single Phase-1 pass; and (iii) the academic two-phase LLM system uses larger base models with substantially more compute than we had access to. Our contribution is not a new state of the art, but a demonstration that the generative SLM paradigm — when combined with retrieval-augmented data generation, structured-output decoding, and character-level ensembling — produces a coherent and reproducible pipeline within the constraints of a public Kaggle environment.

8. Discussion

8.1 What worked, and why

By a wide margin, the largest single intervention in our pipeline was QLoRA fine-tuning, which contributed +0.17 to +0.22 F1 across both individual models and the 3-model ensemble (§7.3). The magnitude of this gain is itself diagnostic: in-context learning with three FAISS-retrieved few-shot examples already exposes the model to the task format and to representative annotations, but three examples cannot encode the full annotation conventions of the dataset — the partial-span boundary style, the handling of negation, and the conventions around paraphrase. Fine-tuning on thousands of pseudo-labelled and gold examples internalizes these conventions in a way that few-shot prompting cannot replicate within a fixed context window. The conservative LoRA configuration we adopted (rank 8, attention-only target modules, learning rate 1×10^{-4} , early stopping on validation loss) was deliberately calibrated against

memorizing noisy pseudo-labels, and the consistent +0.17–0.22 gain across all model scales suggests that the regularisation choice was approximately right.

A second, smaller, but real contribution came from architectural and pretraining diversity in the ensemble. Adding Llama-3.1-8B — from a different model family and a different pretraining corpus than the Qwen3-dominated ensemble — gave +0.022 F1 over the 3-model variant and +0.015 F1 over the strongest single model (§7.4). This is the classical ensemble-diversity argument made concrete: the ensemble benefits not from member count per se, but from members that fail in genuinely different ways. We separately observed that duplicating ensemble members (running the same fine-tuned architecture with a different seed) produced only a within-noise change (≤ 0.001 F1) under our character-level voting scheme, i.e. no meaningful gain.

Third, the per-note XGrammar regex constraint described in §5.5 effectively eliminated the class of failures in which the model emits characters absent from the source note. Because the constraint is structural — the decoder physically cannot sample illegal characters — it provides a stronger guarantee than post-hoc filtering or self-verification. As we noted in §5.5, the constraint is meaningful specifically for generative approaches; discriminative token classifiers face no analogous failure mode.

8.2 What surprised us

Three findings surprised us during the experimental study. The first concerns ensemble composition: the 3-model ensemble (Qwen3-8B + Qwen3-1.7B + LFM2.5-1.2B) actually scored slightly worse than the strongest single model alone (0.710 vs 0.717). Adding a comparatively weaker member dragged the consensus down at the chosen vote threshold — a useful negative result indicating that ensembling is not automatically beneficial and must be calibrated to the strength distribution of its members.

Second, the magnitude of the architectural-fit advantage: even DeBERTa-v3-base — at approximately 184M parameters, smaller than every individual model in our ensemble — outperforms our 4-model LoRA ensemble (~19B parameters in total) by 0.081 F1. Parameter count is not the relevant measure of capability for span extraction; positional inductive bias and full fine-tuning matter more.

Third, the absolute pretrained-only F1 of 0.531 — achieved with no parameter updates whatsoever, relying solely on FAISS-retrieved few-shot prompting — was higher than we anticipated, suggesting that the generative formulation of span extraction is genuinely viable as a zero-fine-tuning baseline. Whether this would hold in a domain less well-represented in pretraining (e.g., non-English clinical notes) is an open question discussed in §9.

8.3 Generalisability and the case for the generative paradigm

Despite our F1 gap relative to DeBERTa, the generative paradigm has a property that the discriminative paradigm lacks: prompt-based adaptability. A real medical NLP system rarely faces a fixed feature taxonomy — rubrics evolve, new clinical features are added, and labeling for new features is delayed. With our pipeline, supporting a new feature requires only updating the prompt and (optionally) adding a few annotated examples to the FAISS retrieval index; the system is functional immediately, with quality comparable to our 0.531 pretrained-only baseline. A DeBERTa pipeline, by contrast, requires a labeled dataset for the new feature, a fine-tuning run, and re-ensembling. The distinction matters in deployment scenarios where rubric changes are frequent and labeled data lags.

This suggests a hybrid usage pattern as the most defensible recommendation from this work: use a fine-tuned discriminative model where labeled data is stable and abundant, and use a generative LLM with retrieval-augmented prompting where the rubric evolves or labels are unavailable. Our experiments demonstrate that the generative path is coherent and reproducible within consumer-grade compute budgets, even when it does not yet match the discriminative ceiling on this specific competition.

9. Limitations and Future Work

9.1 Limitations of this study

We acknowledge several limitations that bound the scope of our claims.

First, compute budget. We did not attempt full fine-tuning of any 8B-parameter base model. Doing so on the combined 39,500-example dataset would have required substantially more GPU memory and wall-clock time than we had access to. As a consequence, our reported gap relative to DeBERTa (which is fully fine-tuned) conflates two factors — paradigm choice (generative vs. discriminative) and fine-tuning depth (LoRA vs. full) — that we could not empirically separate. A fairer paradigm comparison would fully fine-tune both.

Second, pseudo-label quality was not directly measured. Phase 1 produces ~29,600 (note, feature, span) pseudo-labels using Qwen3-8B as the teacher. We measured downstream F1 of models trained on these labels, but did not measure the precision and recall of the pseudo-labels themselves against any gold reference. If the pseudo-labels carry systematic biases (e.g., a tendency to produce minimal spans or to miss negated mentions), those biases are inherited by the student models. A direct evaluation of pseudo-label quality on a small held-out gold subset is a clear gap in the experiments.

Third, inference latency was not quantified. We report F1 scores but no per-row latency measurements. For a real clinical deployment, the comparison between a single DeBERTa forward pass (~tens of milliseconds) and a four-model autoregressive ensemble with up to 512 generation steps each (~seconds) would be decisive. The choice of small (1.2B–8B) base models and the use of tensor parallelism keep our latency manageable on Kaggle T4 hardware, but we did not characterize it.

Fourth, generalisation. All experiments are on English clinical notes from the USMLE Step 2 CS context, and the per-note FSM constraint introduces engineering complexity that may not transfer cleanly to other span tasks. Generalization to non-English clinical settings, other clinical sub-domains (radiology reports, ICU nursing notes), or substantially longer documents is untested; in particular, the FSM size grows with the source note’s character set and may saturate XGrammar’s practical limits for documents beyond a few thousand characters.

9.2 Future work

Several directions could close the gap to DeBERTa or extend the system’s applicability:

(1) Hybrid generative-discriminative pipeline. Use a fine-tuned DeBERTa as a high-recall span proposer, then have an LLM verify or refine the proposed spans. The LLM brings semantic disambiguation and negation handling; DeBERTa brings token-level boundary precision. This combination may be Pareto-optimal with respect to F1 and adaptability.

(2) Larger pseudo-label pool with confidence-based curriculum. Top DeBERTa teams pseudo-labeled the entire ~41,000 unannotated notes; we used 2,000. Scaling Phase 1, combined with a curriculum that introduces high-confidence pseudo-labels first and lower-confidence ones later, would likely close part of the data-scale gap.

(3) Full fine-tuning at 8B scale with adequate compute. Replacing QLoRA with full fine-tuning on the strongest single base model (Qwen3-8B or Llama-3.1-8B) would remove one of the confounds in our DeBERTa comparison, and is the single most informative additional experiment for understanding paradigm choice.

(4) Multilingual extension. Re-running Phase 1 with FAISS examples in the target language and fine-tuning a multilingual base would make the pipeline portable to non-English clinical contexts. The pipeline structure is language-agnostic; only the data and base-model choice are language-specific.

(5) Direct pseudo-label evaluation. Sampling 200–500 pseudo-labeled (note, feature) pairs and re-annotating them by the team would let us report pseudo-label precision and recall directly, isolating teacher quality from student F1.

(6) Threshold tuning. The character-level majority-vote threshold is currently fixed at 3/4. Systematic ablation of thresholds 2, 3, and 4 on the validation set could potentially recover an additional 0.01–0.02 F1, and we leave this as a low-cost future experiment.

10. Conclusion

We set out to test whether a generative small-language-model ensemble, fine-tuned with parameter-efficient methods and equipped with structured-output decoding, could approach the performance of the DeBERTa-based token classification systems that dominate the NBME Score Clinical Patient Notes leaderboard. Our four-model ensemble achieves character-level F1 of 0.732 on the Kaggle public leaderboard (private leaderboard 0.73201) — a roughly fivefold improvement over a TF-IDF + Logistic Regression bag-of-features baseline at 0.140, but 0.081 below a DeBERTa-v3-base baseline at 0.813 and substantially below the best published systems above 0.96. Within the constraints of a public Kaggle T4 environment and parameter-efficient training, the result is a defensible mid-tier outcome that demonstrates the generative paradigm’s viability without claiming superiority over the discriminative paradigm for this task.

Three takeaways from the experimental study warrant emphasis. First, QLoRA fine-tuning was the dominant lever: across both individual models and ensembles, fine-tuning contributed +0.17 to +0.22 F1 over few-shot-only inference, far exceeding the contribution of any other component. Within fine-tuning, conservative configuration choices — rank-8 attention-only LoRA, learning rate 1×10^{-4} , early stopping on validation — mattered more than capacity. Second, ensemble benefit comes from architectural and pretraining diversity, not member count: adding Llama-3.1-8B from a different model lineage to a Qwen3-dominated ensemble produced the +0.015 gain over the best single model, while a 3-model variant without architectural diversity actually scored slightly worse than the best single member. Third, structured-output decoding via per-note XGrammar regex constraints structurally eliminates character-level hallucination, a class of failures specific to generative approaches; this is one of the principal contributions of our pipeline.

A non-algorithmic but practically important lesson came from the engineering pivot from V1 to V2 (§5.6). The original V1 pipeline, validated locally on Qwen3.5-9B and Gemma-4-E4B, was undeployable on the Kaggle inference environment due to a confluence of vLLM dispatch errors and a CUDA version mismatch. The lesson is that deployment constraints — not training constraints — should shape model selection for projects targeting a specific inference environment. End-to-end validation in the target environment is worth doing before committing significant training compute.

If we were to repeat the project, the single change with the highest expected return would be to fully fine-tune an 8B base model instead of using QLoRA. The conservative QLoRA configuration we adopted was correct for the noisy pseudo-label regime, but it leaves a substantial fraction of the model’s capacity unused; combined with the larger pseudo-label pool described in §9, full fine-tuning is the most direct path to closing the gap to DeBERTa within the same generative paradigm. Even at the present gap, however, we believe this work demonstrates that the generative paradigm — supported by retrieval-augmented data generation, parameter-efficient fine-tuning, and structured-output decoding — deserves attention as a coherent and reproducible alternative to discriminative token classification for short-span clinical extraction, particularly in settings where prompt-based adaptability matters more than peak F1.

11. References

- [1] NBME, Score Clinical Patient Notes. Kaggle, 2022. <https://www.kaggle.com/competitions/nbme-score-clinical-patient-notes>

- [2] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. International Conference on Learning Representations. <https://arxiv.org/abs/2006.03654>
- [3] He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. <https://arxiv.org/abs/2111.09543>
- [4] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR. <https://arxiv.org/abs/2106.09685>
- [5] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS. <https://arxiv.org/abs/2305.14314>
- [6] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. <https://github.com/huggingface/peft>
- [7] von Werra, L. et al. (2020). TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>
- [8] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP. <https://arxiv.org/abs/2309.06180>
- [9] Qwen Team, Alibaba Group. (2025). Qwen3 Technical Report. <https://huggingface.co/Qwen/Qwen3-8B>
- [10] Meta AI. (2024). The Llama 3 Herd of Models. <https://arxiv.org/abs/2407.21783>
- [11] Liquid AI. (2024). LFM2.5: Liquid Foundation Models. <https://www.liquid.ai/liquid-foundation-models>
- [12] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. <https://arxiv.org/abs/1702.08734>
- [13] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP. <https://arxiv.org/abs/1908.10084>
- [14] Xu, J., Jiang, Y., Yuan, B., Li, S., & Song, T. (2024). Automated Scoring of Clinical Patient Notes Using Advanced NLP and Pseudo Labeling. arXiv preprint arXiv:2401.12994. <https://arxiv.org/abs/2401.12994>
- [15] Bachmann, M. (2020). RapidFuzz: Rapid fuzzy string matching in Python. <https://github.com/maxbachmann/RapidFuzz>
- [16] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830.
- [17] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS. <https://arxiv.org/abs/2208.07339>
- [18] Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. ICLR. <https://arxiv.org/abs/2003.10555>

12. Acknowledgments

This project builds on a number of open-source libraries and public references. We acknowledge them here in compliance with the course’s plagiarism and code-attribution policy. Open-source libraries used directly in our pipeline include: transformers, peft, trl, and bitsandbytes for fine-tuning; vllm==0.17.1 for inference; sentence-transformers and faiss-cpu for retrieval; rapidfuzz for fuzzy string matching; and pandas / numpy / scikit-learn for data handling and the traditional ML baseline. The pretrained model weights are sourced from the HuggingFace Hub: Qwen/Qwen3-1.7B, Qwen/Qwen3-8B, LiquidAI/LFM2.5-1.2B-Instruct, meta-llama/Meta-Llama-3.1-8B-Instruct, and microsoft/deberta-v3-base.

All figures, tables, and prose in this report were authored by the team. Limited use of an AI assistant (Claude) was made to help brainstorm, write code, proofread section drafts, and cross-check the implementation details in this report against the project’s code repository.

Code availability. Source code and fine-tuned LoRA adapters for all three pipeline stages have been submitted via NTULearn alongside this report. All three pipeline stages are reproducible with the dependencies and hyperparameters detailed in §5.